Exchange Server — Vision & Learning Roadmap

> **Companion to `ExchangeServer_Spec_vN.md`.** That file tracks *what's built and what's broken, session to session*. This one tracks *why this project exists and how I want to build it* — it should change rarely. Upload both together to give an LLM full context in one shot: this document for motivation and working style, the spec for exact code and current bugs.

The Goal

Build a Java stock exchange / matching engine — and use the process of building it to learn the actual engineering disciplines real production exchanges (NASDAQ's INET, LMAX, major crypto venues) are built on: systems programming, concurrency, networking, database/persistence design, distributed systems, and DevOps.

Honest scope note: "build something that works like a real exchange" and "build something at the scale/robustness of a real exchange" are different goals. The second isn't realistically achievable by one person — real venues represent hundreds of engineers and decades of iteration, down to kernel-bypass networking and FPGA-level matching in some cases. The actual goal is the first: build something correct and reasonably serious, and in doing so hit the *same design problems* real systems hit, and understand *why* they were solved the way they were — not necessarily reproduce their scale.

Why This Project

This is a build-to-learn project, not a build-to-ship project. The order of learning matters as much as the end result:

- **Systems programming** — correctness of core logic before anything else
- **Concurrency** — safely handling simultaneous activity once single-threaded correctness is proven
- **Networking** — exposing the engine as a real, reachable service
- **Database management** — durable, crash-safe state without wrecking latency
- **Distributed systems** — what happens when "the exchange" is no longer one machine
- **DevOps** — observability, deployment, and everything that makes a system operable, not just runnable

Guiding Principles (how I want to build and learn)

These have held throughout the project so far and should keep holding:

1. **Reason before code.** Talk through a concrete, numeric scenario in plain English before generalizing to pseudocode or real code. (E.g.: before writing `match()`, trace an incoming SELL for 100 against a resting BUY for 30 by hand, only then generalize to `min(...)`.)
2. **I write the code; the LLM reviews.** I implement, then bring the code for bug-spotting and review — not the other way around.

3. **Verify by tracing, not by asserting.** Before calling any method or design "done," trace it against concrete inputs end-to-end, including edge cases and exit paths — the way `crosses()`, `match()`, `submit()`'s loop, and `removePriceLevel()` were each traced against real numbers before being marked finished.
4. **Call out weak justifications.** If a design choice is being justified with "this is how production systems do it," check whether that's actually true or whether the real reason is simpler (easier to test, avoids duplication, etc.). Don't let "production-grade" become a placeholder for real reasoning.
5. **Check exhaustiveness explicitly.** When enumerating test cases or edge cases, confirm the *dimensions* of the problem (e.g., 2 sides × 3 price relations = 6 cases) rather than listing cases until it feels like enough.
6. **Business logic and data structures stay separated.** Dumb containers (`OrderBook`, `PriceLevel`) expose state; `MatchingEngine` decides what that state *means*. This split should keep showing up as the project grows — e.g. service layer vs. matching core, later on.
7. **Invariants get protected at their source**, not re-checked redundantly by every caller (e.g. quantity validation lives in `fillHeadOrder()` / `Order.fill()`, not recomputed by callers).

The Roadmap

Phases are listed in expected order, but this is expected to evolve — it reflects current best understanding, not a locked plan.

1. Systems Programming — Correctness (current phase)

Single-threaded, in-memory, price-time-priority matching core. Domain modeling, the `submit()` / `match()` / `crosses()` design, JUnit tests, a runnable `Main.java`. Nothing here should depend on a framework — this stays plain Java throughout.

2. Concurrency

Move from "one order at a time, no races" to safely handling orders arriving close together in real time. Central question to resolve when this phase starts: does an individual order book need internal thread-safety, or does correctness come from funneling all activity for one instrument through a single-threaded engine, with concurrency only at the boundary across *different* instruments? This connects directly to how the service layer ends up sharding work.

3. Networking — Service Layer

Expose the engine over Spring Boot (or similar). This is additive, not a rewrite — the matching core from Phase 1 shouldn't need to change to be wrapped in a REST controller. Likely shape: a small number of services around a non-distributed matching core per instrument ("microservices at the edges, monolithic core in the middle"), rather than splitting the matching logic itself across services.

4. Persistence / Database Management

Durable order and trade history, crash recovery. The interesting design problem is keeping this off the hot matching path — real systems write to durable storage asynchronously so persistence doesn't add latency to matching itself.

5. Distributed Systems

Replication, failover, and (if multiple nodes need to agree on shared state) consensus. Flagged honestly: the hardest phase, the most theory-heavy, and the one where "match real exchange scale" is least realistic solo — but the concepts, and the reasons distributed matching engines are avoided in practice, are very learnable regardless.

6. DevOps / Observability

Metrics, logging, deployment, monitoring. Likely the most transferable skill of the whole roadmap, independent of this specific domain.

Relationship to the Spec File

- **This document** — stable, changes rarely, captures motivation and working style.
- `ExchangeServer_Spec_vN.md` — changes every session, captures exact current code, known bugs, and the immediate next task.

Upload both when resuming a session. If only one can be uploaded, the spec file is more urgent for continuing exactly where the code left off; this file matters most when starting a new phase, or when a new LLM/session needs the full "why" behind a design choice.

Instructions for the Assisting LLM

If you're an LLM reading this at the start of a session:

- Adopt the working style in **Guiding Principles** above — this is the actual, tested collaboration style across many sessions, not an aspirational description.
- Default to Socratic pacing: pose a concrete scenario, let the user reason through it, only confirm or correct — don't jump straight to the answer or the code.
- The user writes implementation code themselves. Your role is review, bug-spotting, and design discussion, not authoring the implementation.
- When a claim about a method or design being "correct" or "finalized" comes up, check it by tracing a concrete example — numbers, not abstractions — rather than accepting or asserting it at face value.
- Treat the roadmap phases as a guide to what's next, not a rigid gate — but don't assume later phases are needed to make earlier ones meaningful. Each phase should stand on correct work from the phase before it.
- Update this document's roadmap section only when a phase genuinely completes or the plan itself changes — not every session.